Module Name: Selenium WebDriver Setup - Environment Verification
Test Case ID: st_13
Tester Name: Aman Sevak
Webpage URL: https://www.selenium.dev

- ## Scenario ID: 1a

Scenario: Check Python interpreter in PyCharm
Test Steps:

1. Open PyCharm
2. Go to New Project and enter name SoftwareTesting
3. Select the available Python interpreter
4. Click on Create

Screenshots:
-

- ## Scenario ID: 1b

Scenario: Verify Selenium installation
Test Steps:

1. Open Terminal in PyCharm
2. Enter command: pip install selenium
3. Wait for installation to complete
4. Enter command: pip show selenium
5. Check the version details displayed

Screenshots:
-

- ## Scenario ID: 1c

Scenario: Write and save the automation script
Test Steps:

1. Right-click on project and create tutorial13.py
2. Import webdriver from selenium
3. Add code to open Chrome and load the website
4. Save the file

Source Code:

```
from selenium import webdriver

driver = webdriver.Chrome()

driver.maximize_window()

driver.get("https://www.selenium.dev/")

print("Browser launched successfully")
print("Page Title:", driver.title)
print("Current URL:", driver.current_url)

driver.quit()
```

Screenshots:

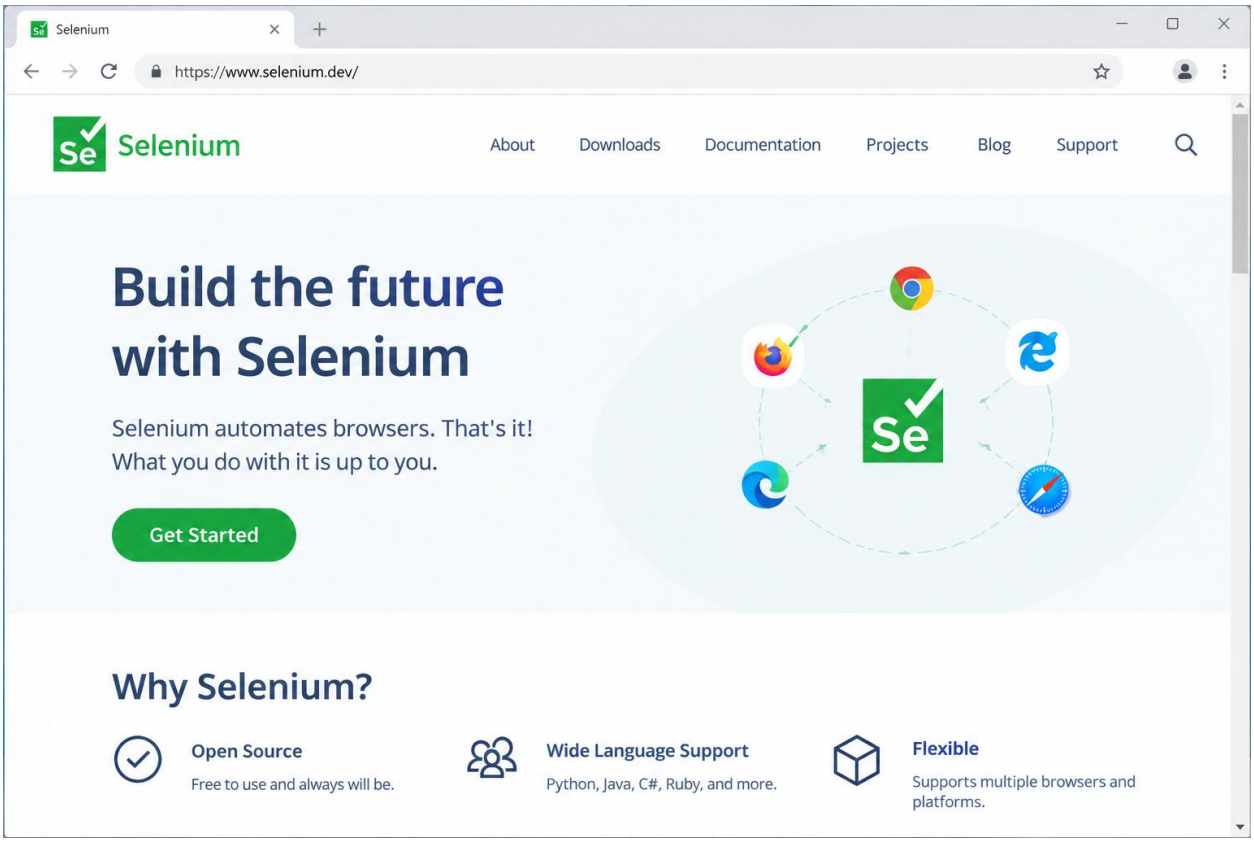
-

- ## Scenario ID: 1d

Scenario: Launch Chrome and open the Selenium website
Test Steps:

1. Right-click on tutorial13.py
2. Click on Run 'tutorial13'
3. Chrome browser opens and window gets maximized
4. Website https://www.selenium.dev loads in the browser

Screenshots:



- **Scenario ID: 1e**

Scenario: Validate console output and close browser
Test Steps:

1. Check the PyCharm Run console
2. Verify the printed Page Title and Current URL
3. Confirm that the browser closes automatically (driver.quit)

Expected Output:

```
Browser launched successfully
Page Title: Selenium
Current URL: https://www.selenium.dev/
```

Screenshots:

-

Summary Of Test Results:

Total Test Cases- 5

Passed- 5
Failed- 0
Pass %- 100%

- ## Conclusion and Recommendations:

Pass Rate: 100%, Environment Ready for Automation Testing

- ## Issues Found

No issues found.

- ## Recommendations

1. **Add Explicit Waits**
   Use WebDriverWait in future scripts so that the page is fully loaded before reading its title or elements.
2. **Use a Virtual Environment**
   Create a separate Python virtual environment for each project to avoid library version conflicts.

- ## Bug Reports:

No bugs were found during execution of this practical. All test cases passed.